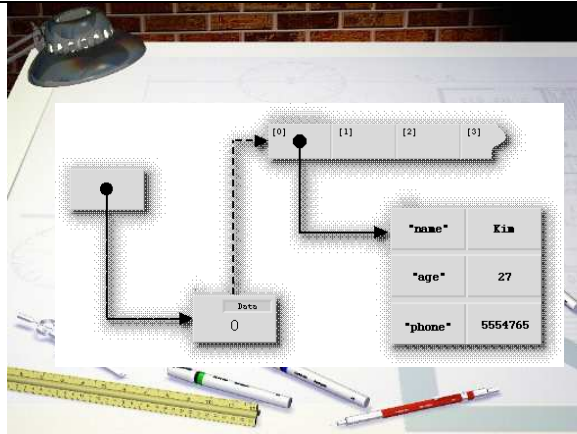




Slide 1

CS 170

Java Programming 1

Classes and Arrays

Duration: 00:01:02

Advance mode: Auto

Notes:

Hi There. Welcome to the CS 170, Java Programming 1 lecture, Classes and Arrays.

In previous lectures you learned how to work with partial arrays; filling them with input, inserting and deleting elements, and even resizing them. When you use a partially-filled array, such as those we employed for insertion and deletion, you need to keep track of the number of elements in use separately. That sounds like a job for encapsulation and a class, doesn't it?

In this lecture, we'll focus on writing classes to hide some of the complexity of manipulating partially-filled arrays. To get started, import the Eclipse project, ic19 from the ic19.zip file you'll find on the class Web site immediately below the link for this lecture. While you're at it, start your IC exercise document for this week, and create a new section of this lecture's exercises.

The SimpleArrayClass

- Partially-filled arrays are useful, but a little complex
 - Often look for an easier way to get the job done
- Use **encapsulation** to **hide** the details inside a class
 - SimpleArrayClass** uses techniques from **InputArray**
- Step 1:** Convert variables to instance variables

```
12: /*
13: public class SimpleArrayClass
14: {
15:     // instance variables
16:     private int size = 0;
17:     private int capacity = 100;
18:     private double[] ar = new double[capacity];
19: }
```

Slide 2

The SimpleArrayClass

Duration: 00:01:10

Advance mode: Auto

Notes:

If you look back at the program InputArray from the previous lesson, you'll see that we used a partially-filled array of double values, along with two additional variables, capacity and size, to keep track of how many elements are actually used. The only problem is, the code is a little complex, and so you're likely to look for an easier way to get the job done when a partially-filled array would be the ideal solution.

That's where the OOP principle of encapsulation comes to the rescue; we can hide all of those messy details behind an easy to use interface. That's what the SimpleArrayClass program does. It uses the techniques from the InputArray program, but applies them in terms of object-oriented programming. Here's how.

Step 1 is to just take the variables we used in the InputArray program, and turn them into instance variables. In addition to the two variables for size and capacity, we also need a variable to hold the array elements themselves. Here's what that should look like.

SimpleArrayClass Methods

- **Step 2:** methods instead of tracking **size** or **capacity**

```
20:  /**
21:   * @return true if the array is full
22:   */
23:  public boolean isFull()
24:  {
25:      return size >= capacity;
26:  }
27:
```

- User doesn't need to know **how** data is stored
- Just asks "is the array full, or can I add another element?"

- Defensive programming?

Slide 3

SimpleArrayClass Methods

Duration: 00:01:03

Advance mode: Auto

Notes:

Step 2 is to provide methods for all of the normal operations that you would want to perform on the array. Instead of the user keeping track of how many elements there are, the class can do it, by providing a method like `isFull()` shown here.

When you do this, the user doesn't need to know that the variable `size` exists; all they need to know is "can I add an element to this array, or is it full?".

Note that in the `isFull()` method, I've checked to see if `size >= capacity`, when, technically, the array is full when `size == capacity`. This is an example of defensive programming; making sure that if I make a mistake elsewhere, accidentally setting `size` greater than `capacity`, it won't crash my program. I'm not a great fan of defensive programming, though, because it misleads those who read the code into believing that `size` could have a valid value greater than `capacity`, which is not the case.

Adding Array Elements

- Array variable is hidden, so we need an **add()** method

```
34:  public boolean add(double value)
35:  {
36:      if (! isFull())
37:      {
38:          ar[size] = value;
39:          size++;
40:          return true;
41:      }
42:      return false;
43:  }
```

- 1. Checks to see if there's still room in the array
- 2. Add element if room, update **size**, return **true**
- 3. If there's no room, then return **false**

- User doesn't need to know about **size** or **capacity**

Slide 4

Adding Array Elements

Duration: 00:00:59

Advance mode: Auto

Notes:

Because the actual array is a private instance variable, the elements are "hidden" from the users of the `SimpleArray` class. To allow the user to add an item to the array, we can write an `add()` method, that works like this: Note that the method:

1. Checks to see if there is still room in the array by using the `isFull()` method. (Note the use of the NOT operator in the condition.)
2. If the array isn't full, then we simply add the element at the correct position, (using the `size` instance variable), update the variable, and return `true` to signal that the element was added successfully.
3. If there wasn't enough room for the element, then the method returns `false` and doesn't modify either the array or the other instance variables.

As you can see, the user of your class doesn't have to know about the `size` and `capacity` variables to use the class at all.

Displaying Array Elements

- Can't access array elements, so we can't display them
 - Add a **get()** method to retrieve specific values
 - Can override **toString()** to print the entire array

```
44:  /**  
45:   * Prints the valid elements in this array  
46:   */  
47:  public String toString()  
48:  {  
49:      String ret = "(";  
50:      if (size == 0)  
51:      {  
52:          ret += ")";  
53:      }  
54:      else  
55:      {  
56:          for (int i = 0; i < size - 1; i++)  
57:              ret += ar[i] + ", ";  
58:          ret += ar[size-1] + ")";  
59:      }  
60:      return ret;  
61:  }
```

Slide 5

Displaying Array Elements

Duration: 00:00:33

Advance mode: Auto

Notes:

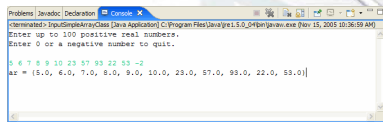
To display the individual values, you'd normally write a **get()** method to allow the user to extract a particular value. I'll leave that as "an exercise for the reader". Instead, for our example here, let's just overload the **toString()** method to print the entire array, element by element, like this

The version of **toString()** shown here returns a representation similar to one that would be used to initialize the array in the first place.

Using SimpleArrayClass

- **Exercise 1:** open **InputSimpleArrayClass**
 - Same program, but we'll use our class
 - **Step 1:** Create an instance of the class
 - **Step 2:** Use a class method to control the loop
 - **Step 3:** Use a class methods to input data elements
 - **Step 4:** Use a class method to print data

- Run and snap a picture



Slide 6

Using SimpleArrayClass

Duration: 00:02:26

Advance mode: Auto

Notes:

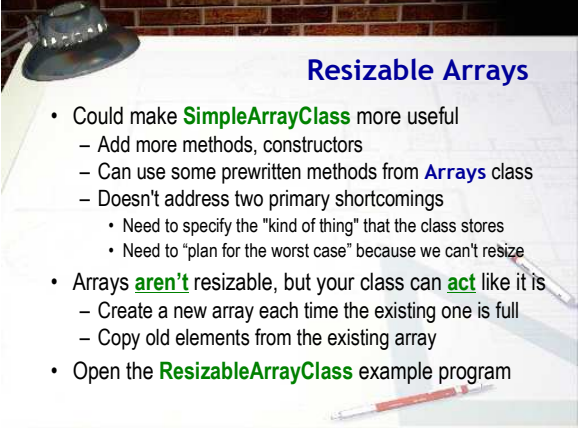
To test the **SimpleArrayClass**, let's recreate the **InputArray** program from the last lesson. The program **InputSimpleArrayClass.java** will use the class you just wrote, instead of a plain-old-array, to hold up to 100 numbers input by the user, and then print them out. Go ahead and open the program in Eclipse and locate the **main()** method.

This version of the program creates a **Scanner** object to capture the user's input, just as the previous one did, but then, instead of creating an array, it asks you to create a **SimpleArrayClass** object named **ar** for Step 1.

Next, we ask the user to input 100 real numbers. In the earlier program, we used the length of the array to tell the user how many numbers to enter; which the **SimpleArrayClass** though, we can't do that, because the internal array is private, and we didn't provide a **length()** method, (although we easily could have).

The main loop is similar to the earlier program, with these differences:

- The starter code is an endless loop that continues while true. For Step 2, modify that to use a method from **SimpleArrayClass** as the loop bounds. Here's a hint: if **ar** is not full, then we should enter the loop. This, you'll recall, is the loop's necessary bounds.
- Inside the loop, we read the value, using the **Scanner** object, and then check to see if the user wants to exit the loop by entering zero or a negative number. If they do, we use **break** to exit the loop. This is the intentional bounds in this "loop and a half".
- At the bottom of the loop, for Step 3, use one of the **SimpleArrayClass** methods to add the number to the array.
- After the loop ends, at Step 4, instead of using another loop to print the contents of the array, (something that we

	couldn't do anyway, given the design of our class), just use concatenation to print the contents of <code>ar</code>; the <code>toString()</code> method added to <code>SimpleArrayClass</code> provides the formatting. Go ahead and run the program and snap a picture for Exercise 1. Below the picture, paste in the code you added to get it to work.
 Resizable Arrays • Could make SimpleArrayClass more useful – Add more methods, constructors – Can use some prewritten methods from Arrays class – Doesn't address two primary shortcomings • Need to specify the "kind of thing" that the class stores • Need to "plan for the worst case" because we can't resize • Arrays aren't resizable, but your class can act like it is – Create a new array each time the existing one is full – Copy old elements from the existing array • Open the ResizableArrayClass example program Slide 7 🎧 Resizable Arrays Duration: 00:02:01 Advance mode: Auto	Notes: You could easily continue to expand <code>SimpleArrayClass</code>, adding methods to access, insert, find, sort, and delete elements, often simply by employing the "built-in" methods supplied by the <code>Arrays</code> class. You'd certainly want to add a constructor, so users could create <code>SimpleArrayClass</code> objects using sizes other than 100. But, while the <code>SimpleArrayClass</code> hides some of the complexity behind using partially-filled arrays, it doesn't address two other problems: • <code>SimpleArrayClass</code> can only store doubles, while regular arrays can store any type. • You still need to plan for the worst case when deciding how big to make your array. Because the size of an array is fixed, once it's been created, the user has to rely on the <code>isFull()</code> method to know if it's safe to add elements to the array, or to check the value returned from the <code>add()</code> method, to make sure that the element was added to the array. Even though arrays aren't resizable, though, that doesn't mean that your class has to act that way. By automatically creating a new, larger internal array each time the existing array fills up, and then copying the elements from the existing array into the new storage, you can make your array class appear to the user as a resizable array. Let's look at an example that does that. The <code>ResizableArrayClass</code> program is very similar to the <code>SimpleArrayClass</code> with a few changes. Go ahead and open it in Eclipse and we'll look at some of those changes. The first change you should notice is instead of setting capacity to 100, this class sets it to 1. (You wouldn't do this in "real life". Instead, you'd start with some more reasonable starting value; 10, 16, and 32 are often used). I used such a small size to illustrate how the class resizes itself.

Adding and Expanding

- If array is full when adding, then expand the array
- To expand, increase capacity, create a new array, copy old elements to the new array, and point your array variable to the new memory

```
38:
39:  /**
40:   * Increases the length of the array by 1
41:   */
42:  public void expand()
43:  {
44:      capacity++;
45:      double[] temp = new double[capacity];
46:      System.arraycopy(ar, 0, temp, 0, size);
47:      ar = temp;
48:  }
49:
```

Slide 8

Adding and Expanding

Duration: 00:01:01

Advance mode: Auto

Notes:

Because the array is now resizable, there's no need to return a "success report" from the add() method; we can change the return type to void. Inside the method, we still need to check and see if the array is full. When it is, however, instead of returning false to indicate that the element couldn't be added, we just call the internal expand() method, and then insert the element in the new space that's created

To expand the array, the method increments capacity, allocates a new array, copies the old array elements into the new array, and then assigns the instance variable ar to "point to" the newly allocated (and larger) memory.

In this case, I've used a static method in the System class, arraycopy(), to copy the values from the old array elements into the new array elements. Before we put ResizableArrayClass to work, let's look a little closer at the concept of copying arrays.

Bad Array Copies

- When we expand the array we need to make a **copy** of the existing array. How do we do that?
 - We may need a copy to pass to a modifying method
- **Method 1:** The wrong way

```
int [ ] copy = nums;
doubleIt(copy);
```

- Variable **copy** refers to the same elements as **nums**
 - The values in **nums** array are still modified

Slide 9

Bad Array Copies

Duration: 00:00:52

Advance mode: Auto

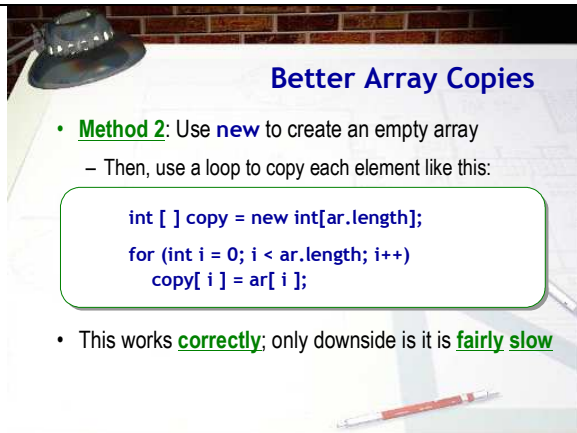
Notes:

When we expand our existing array, we need to make a copy of the existing array elements. Often, you'll need to make a copy of an array when you need to pass it to a method that modifies the array, such as the doubleIt() method shown in this lecture, but you still need to preserve the original array contents as well.

This is not hard to do, but you have to know how to do it. You cannot use the assignment operator like this:

```
int[] copy = nums;
doubleIt(copy);
```

If you do this, instead of having two array variables referring to the same actual array (copy and nums), you now have three: nums, copy, and the formal parameter in the doubleIt() method. Any changes made to the elements of the parameter passed to the method doubleIt() apply to both copy and nums as well. The elements in nums are modified as well.



Better Array Copies

- **Method 2**: Use **new** to create an empty array
 - Then, use a loop to copy each element like this:

```
int [ ] copy = new int[ar.length];
for (int i = 0; i < ar.length; i++)
    copy[ i ] = ar[ i ];
```

- This works **correctly**; only downside is it is **fairly slow**

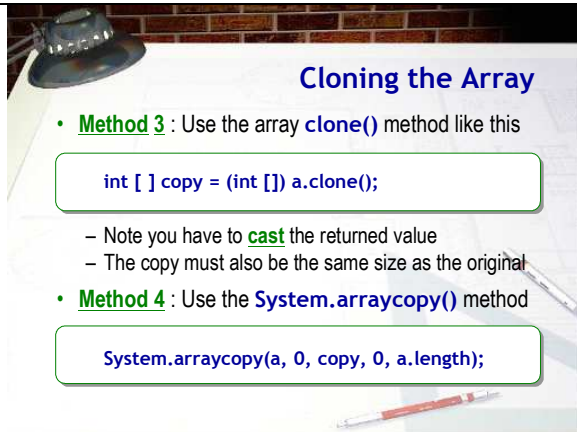
Slide 10
Better Array Copies
Duration: 00:00:36
Advance mode: Auto

Notes:

Since the assignment method obviously doesn't work, how do you do this correctly? There are three ways.

First, you can just allocate an array containing the same number of elements as your original, and then, write a loop and copy each element one-by-one from the original array to the new array.

This works just fine and is perfectly correct. The only downside is that it really is fairly slow when you have a large number of elements, such as those you'll encounter when processing the pixels in a picture.



Cloning the Array

- **Method 3**: Use the array **clone()** method like this

```
int [ ] copy = (int []) a.clone();
```

- Note you have to **cast** the returned value
- The copy must also be the same size as the original

- **Method 4**: Use the **System.arraycopy()** method

```
System.arraycopy(a, 0, copy, 0, a.length);
```

Slide 11
Cloning the Array
Duration: 00:01:58
Advance mode: Auto

Notes:

Methods three and four both make use of optimized code available on your CPU, designed specifically for quickly moving bits from one location in memory to another.

Method 3 uses the array **clone()** method to both allocate the elements and fill them in. The **clone()** method just takes a snapshot of the raw memory used for your array and duplicates it. There are two downsides to using this method:

- First, you have to cast the returned value to the type of variable you'll store it in. That's what the **int[]** in parentheses appearing before the words **a.clone()** is for.
- Second, you can only clone an entire array. You can't just copy some of the elements or copy the elements into a new array that's a different size. You do, indeed, get a clone. That's why we couldn't use it for the **ResizableArrayClass**, since each copy needs to be one element larger than the original.

Finally, for method 4 you can use the **arraycopy()** method in the **system** class. This method takes five parameters.

- The array to copy from
- The element to start copying from. In my slide here I'm copying from position 0, or the beginning of the array.
- The array to copy to
- The element to start copying into in the destination array. This allows you to place a set of elements in a new position when they are copied.
- Finally, the number of elements to copy.

System.arraycopy() uses your CPU's native code to move the elements from one array to another. Unlike the **clone()** method, though, **arraycopy** does not allocate the destination array for you. You have to do that yourself.

Using ResizableArrayClass

- **Exercise 2:** open **InputResizableArrayClass**
 - Same program, but we'll use the new class
 - Program is both simpler and shorter
 - **Step 1:** Create an instance of the class
 - **Step 2:** Use a class methods to input data elements
 - **Step 3:** Use a class method to print data
- Run and snap a picture

Slide 12

Using ResizableArrayClass

Duration: 00:01:39

Advance mode: Auto

Notes:

OK, now it's your turn again. Let's do one last exercise for this lecture. Open the file `InputResizableArrayClass.java`. This is the same program you just created, but instead of `SimpleArrayClass` we'll use `ResizableArrayClass`. Here are the changes you should make.

For Step 1. create a `ResizableArrayClass` object, rather than a `SimpleArrayClass` object, as you'd expect.

Instead of telling the user to enter 100 numbers, (and adding a corresponding loop test), the program prompt asks them to enter any number of positive integers. The program will no longer stop when a particular number of input values have been entered.

Rather than using a "loop-and-a-half", with the necessary bounds and the if statement controlling the user's exit decision, we can use a simpler inline test to read the user's input, store it in a variable, and check it against zero, all on one line. >

For Step 2, add a single statement to the loop body that adds the number to the resizable array.

For Step 3, the printout works just the same way, (as do the other parts of the program.) Just concatenate your object to display the results. Here's a screenshot of a sample run of the program.

Run the program, snap a picture and paste it into your IC document. Underneath the picture, paste the code for your program.